«Конструктор виртуального мира на основе ООП»

Общая идея проекта

Во втором проекте необходимо **переработать проект 1**, в котором объекты виртуального мира создавались в основном с помощью переменных, массивов, условий и случайных значений, и реализовать его уже в **объектно-ориентированном стиле на Java**.

Теперь программа должна не просто генерировать описания, а работать с **классами и объектами**, использовать **наследование, абстракцию, инкапсуляцию, перегрузку, статические поля, обработку исключений и запись данных в файл**.

Пользователь запускает программу и поочерёдно формирует прототип виртуального мира:

- **выбирает** или создаёт **персонажа**;
- **выбирает** или создаёт **питомца**;
- **выбирает** или создаёт **предмет**;
- **выбирает** или создаёт **событие**;
- **выбирает** или создаёт **другой объект мира** по выбору разработчика.

После этого программа выводит итоговое описание виртуального мира и сохраняет созданные объекты в **файл**.

Цель проекта

Закрепить навыки объектно-ориентированного программирования на Java на практическом примере.

Учебные цели

В ходе выполнения проекта необходимо показать умение:

- проектировать программу с использованием **классов и объектов**;
- применять **абстрактный класс**;
- использовать **наследование**;
- скрывать данные через **private-поля** и предоставлять доступ к ним через **геттеры и сеттеры**;
- реализовывать **абстрактные методы** и переопределять их в дочерних классах;

- использовать **конструкторы**, в том числе **перегруженные**;
- использовать **перегрузку методов**;
- применять **статические поля**;
- обрабатывать ошибки с помощью **try-catch**;
- выполнять **запись данных в файл**;
- организовывать код в виде нескольких классов и методов.

Постановка задачи

Необходимо создать консольную программу **«Конструктор виртуального мира на основе ООП»**.

Программа должна позволять пользователю формировать виртуальный мир из нескольких объектов. Основной объект – **персонаж** – должен быть реализован через **абстрактный класс и наследование**. Остальные объекты мира должны быть реализованы через обычные классы, но также должны поддерживать выбор из заранее подготовленных вариантов и возможность пользовательского создания.

Обязательная структура проекта

1. Абстрактный класс для персонажей

Для объектов типа **персонаж** необходимо создать **абстрактный класс**.

В абстрактном классе должны быть:

- **закрытые поля** (private);
- **открытые методы доступа** (get / set);
- **конструкторы**;
- **абстрактные методы**;
- при необходимости – обычные методы с готовой реализацией.

Пример возможных полей абстрактного класса:

- name
- race
- heroClass
- health
- attack
- intelligence
- luck

Пример абстрактных методов:

- generateDescription()

- getSpecialAbility()

2. Класс-наследник для персонажа

На основе абстрактного класса необходимо создать **класс-наследник**, *например*, Character.

Именно объекты этого класса создаются в программе.

В классе-наследнике должны быть:

- реализация абстрактных методов;
- собственные поля при необходимости;
- конструкторы;
- перегруженные методы или конструкторы;
- удобный вывод информации о персонаже.

3. Заготовленные персонажи

В программе должно быть создано **5 заранее подготовленных персонажей**.

Для каждого персонажа должно быть задано описание, *например*,

- имя;
- раса;
- класс;
- характеристики;
- особенность;
- краткое сюжетное описание.

Требование

Пользователь должен:

- увидеть список готовых персонажей;
- получить описание каждого;
- выбрать одного из них.

Дополнительно

Если ни один шаблон пользователю не подходит, программа должна предоставить возможность **создать собственного персонажа вручную**.

4. Остальные объекты виртуального мира

Кроме персонажа, программа должна работать ещё минимум с **3-4 объектами виртуального мира**. Например,

- питомец;

- таинственный предмет;
- событие;
- место;
- блюдо;
- музыкальная группа;
- транспорт;
- фракция и др.

Для этих объектов **необязательно использовать абстрактный класс**, но обязательно использовать **классы, объекты, поля, методы, конструкторы**.

Для каждого такого объекта должно быть:

- минимум **3-5 заранее подготовленных заготовок**;
- возможность выбора одной из заготовок пользователем;
- возможность при желании создать свой вариант вручную.

Пример пользовательского сценария

Этап 1. Выбор персонажа

Программа выводит 5 готовых персонажей с краткими описаниями.

Например,

Аэлар – маг из северных земель, владеющий ледяными чарами.

Роксана – охотница на чудовищ, выросшая в пограничных землях.

Тарг – изгнанный воин древнего клана.

Лиора – лекарь и знаток запретных рун.

Каэль – механист из подземного города.

Пользователь:

либо выбирает номер шаблона;

либо вводит команду создания собственного персонажа.

Этап 2. Выбор питомца

Пользователь выбирает готового питомца или создаёт собственного.

Этап 3. Выбор предмета

Пользователь выбирает заготовленный артефакт или создаёт свой.

Этап 4. Выбор события

Пользователь выбирает одно из событий или задаёт своё.

Этап 5. Итог

Программа формирует краткий сценарий виртуального мира и сохраняет результат в файл.

Что обязательно должно быть реализовано

1. Инкапсуляция

Во всех основных классах данные должны быть скрыты через private-поля.

Доступ к ним должен осуществляться через:

геттеры;

сеттеры.

2. Абстракция

Для персонажей обязательно должен использоваться **абстрактный класс**.

3. Наследование

Класс персонажа должен быть **наследником абстрактного класса**.

4. Конструкторы

В программе должны быть:

обычные конструкторы;

перегруженные конструкторы.

Например,

конструктор с полным набором параметров;

конструктор с неполным набором параметров;

конструктор по умолчанию.

5. Перегрузка методов

В проекте должна быть реализована **перегрузка хотя бы одного метода**.

Например,

метод создания описания в двух вариантах;

метод вывода объекта в краткой и полной форме;

метод сохранения в файл с разными параметрами.

6. Статическое поле

В проекте должно использоваться хотя бы одно **статическое поле**.

Например,

счётчик созданных объектов;

общее количество персонажей;

название мира, общее для всех;

номер следующего объекта.

7. Обработка исключений

Программа должна корректно обрабатывать ошибки пользователя и **ошибки при работе с файлами**.

Обязательно использовать try-catch.

Какие ошибки желательно обработать:

- ввод строки вместо числа;
- выбор несуществующего пункта меню;
- ошибка создания или записи файла;
- пустой ввод имени или названия объекта.

8. Работа с файлом

Все созданные или выбранные пользователем объекты должны быть **сохранены в файл**.

Требование

В файл должно записываться:

- описание персонажа;
- описание остальных объектов;
- итоговый сценарий виртуального мира.

Формат файла

Допускается обычный текстовый файл .txt.

Например,

world.txt

virtual_world.txt

9. Методы

В проекте должно быть не менее **5 собственных методов**, не считая геттеров и сеттеров.

Например,

- метод вывода меню;
- метод выбора готовой заготовки;
- метод создания собственного объекта;
- метод генерации итогового сценария;

- метод записи в файл.

Минимальный набор классов

Ниже приведён рекомендуемый минимум классов.

Обязательные классы:

Абстрактный класс для персонажей

Character – класс-наследник

Pet

Artifact или **Item**

Event

Main – запуск программы

Дополнительно можно добавить:

World

Location

Faction

FileManager

Требования к пользовательскому интерфейсу

Программа должна быть **консольной** и понятной для пользователя.

Необходимо

- выводить понятные инструкции;
- показывать меню выбора;
- сообщать об ошибках;
- подтверждать успешное создание объекта;
- **красиво** выводить итоговый результат.

Требования к содержанию заготовок

Для персонажей. Необходимо подготовить **5 уникальных персонажей**. Они должны отличаться не только именем, но и:

- расой;
- классом;
- набором характеристик;
- историей;
- особенностью.

Для остальных объектов. Необходимо подготовить несколько осмысленных заготовок, а не случайный набор слов.

Например, для питомцев:

сумеречный лис;

механический ворон;

кристальный дракончик;

болотный дух;

песчаный кот.

Для предметов:

обсидиановый амулет;

светящийся компас;

клинок забытого ордена;

поющий кристалл;

часы, искажающие время.

Итог работы программы

После завершения выбора программа должна сформировать **связное текстовое описание виртуального мира**, например,

В мире Этерион появился герой Каэль – механист из подземного города. Его спутником стал механический ворон, способный распознавать скрытые механизмы.

Герой владеет древним предметом – поющим кристаллом, который реагирует на приближение опасности.

В день прибытия героя в столицу произошло редкое событие: над городом открылся световой разлом.

Так начинается история нового виртуального мира.

Чек-лист

- Консольная программа на Java.
- Использование Scanner для ввода данных.
- Использование нескольких классов.
- Для персонажей – **абстрактный класс**.
- **Private-поля** во всех основных классах.
- **Геттеры и сеттеры**.
- **Конструкторы**.
- **Перегрузка конструктора**.

- **Перегрузка метода**.
- **Статическое поле**.
- **Объект класса-наследника**.
- **5 заранее подготовленных персонажей**.
- Возможность выбора готового персонажа.
- Возможность создания собственного персонажа.
- Аналогичная логика выбора или создания для других объектов мира.
- **Обработка исключений**.
- **Запись результатов в файл**.
- Не менее **5 собственных методов**.
- Читаемый, аккуратный и логично организованный код.
- Осмысленное и креативное наполнение виртуального мира.

Ограничения и рекомендации

Не использовать графический интерфейс – только консоль.

Не делать проект слишком примитивным: заготовки должны быть интересными.

Не сводить программу к простому выводу текста без использования ООП.

Код должен быть разбит на классы, а не написан целиком в main.

Желательно избегать дублирования кода.